

A DUAL-CNN DEEP LEARNING FRAMEWORK FOR COCOA CROP PATHOGEN DIAGNOSIS & SUSTAINABLE RECOMMENDATION SYSTEM

A Comprehensive Project Dissertation and Research Report



Submitted By: Gowraww

Degree: B.Tech / M.Tech

Institution: Department of Computer
Science

Year: 2026

Certificate of Originality

This is to certify that the project report entitled '**A Dual-CNN Deep Learning Framework for Cocoa Crop Pathogen Diagnosis & Sustainable Recommendation System**', submitted by **Gowraww** in partial fulfillment of the requirements for the award of the Degree, is a record of bonafide project work carried out under supervision.

The results embodied in this report have not been submitted to any other University or Institute for the award of any degree or diploma. To the best of our knowledge, the code implementation and research methodologies are original and represent diligent agricultural research.

Internal Supervisor

Department of Computer Science & Engineering
Agri-Tech Research Cell

Head of Department

School of Technology and Agriculture

Acknowledgments

I express my deep sense of gratitude to my project supervisor and mentors for their invaluable guidance, constant encouragement, and critical feedback throughout this agricultural computer vision research.

I am highly indebted to the Department of Agricultural Extension Services and local farming cooperatives in Sullia, Karnataka, for providing access to raw cocoa crop leaf and pod datasets. Without their agricultural domain knowledge, this project's recommendation system would not have been possible.

Lastly, I thank my family and peers for their continuous support and patience during the intensive model training, debugging, and compilation of this 50-page dissertation report.

Abstract

Deep Learning for Cash Crop Diseases

Tropical cash crops like Cocoa (*Theobroma cacao*) play a massive role in global economic trade. Fungal and pest outbreaks, particularly Black Pod Rot and Pod Borer, degrade crop quality and cause up to 40% crop losses annually. Traditional disease identification is slow, expensive, and error-prone.

This dissertation proposes a sequential, dual-model Convolutional Neural Network (CNN) framework integrated into a Flask-based agricultural web app. Stage 1 employs a binary detector to filter irrelevant uploads, achieving ~92% validation accuracy. Stage 2 runs a 4-class classifier to identify specific diseases (Black Pod Rot, Healthy Borer, Pod Borer, Healthy). The app links crop diagnoses directly to official channels (IFFCO Bazar and Karnataka Raita Mitra) to provide farmers with genuine, subsidized fertilizer support.

The model's performance has been verified programmatically and validated using real-world datasets. This report outlines the dataset preprocessing, deep learning layers, web integration, and agricultural science behind the recommendation engine.

Table of Contents

Chapter Index (1/2)

Chapter 1: Introduction

- 1.1 Background of Cocoa Cultivation Page 7
- 1.2 Global Economic Importance of Cocoa Page 8
- 1.3 Pathogen Vulnerability and Farming Risk Page 9
- 1.4 Traditional Identification Constraints Page 10
- 1.5 Project Motivation & Scope Page 11
- 1.6 Objectives of the Study Page 12
- 1.7 Thesis Outline Page 13

Chapter 2: Literature Review

- 2.1 Classical Crop Surveillance Methods Page 14
- 2.2 Image Processing in Precision Agriculture Page 15
- 2.3 Hand-crafted Feature Extraction vs Deep Learning Page 16
- 2.4 Convolutional Neural Network (CNN) Foundations Page 17
- 2.5 Comparative Analysis of Modern CNNs Page 18
- 2.6 Plant Pathology Automation Trends Page 19
- 2.7 Open Research Gaps & Input Noise Vulnerabilities Page 20

Table of Contents

Chapter Index (2/2)

Chapter 3: Dataset Analysis & Data Engineering

- 3.1 Dataset Overview & Class Distributions Page 21
- 3.2 Imbalance Challenges & Class Weighting Math Page 22
- 3.3 Data Augmentation Engineering Page 23
- 3.4 Preprocessing Pipeline & Normalization Page 24

Chapter 4: Methodology and System Design

- 4.1 Sequential Dual-CNN Gate Pipeline Page 25
- 4.2 Mathematical Theory of Convolutions Page 26
- 4.3 Pooling and Dimensionality Reduction Page 27
- 4.4 Activation Functions: ReLU, Sigmoid, Softmax Page 28
- 4.5 Optimization Algorithms & Loss Functions Page 29

Chapter 5: Detailed Model Architectures

- 5.1 Model 1 Layer Breakdowns Page 30
- 5.2 Model 2 Layer Breakdowns Page 31
- 5.3 Hyperparameter Settings Page 32
- 5.4 Regularization & Early Stopping Page 33

Table of Contents

Chapter Index (3/3)

Chapter 6: Experimental Setup & Results

- 6.1 Hardware and Software Infrastructure Page 34
- 6.2 Model 1 Detector Training Curves Page 35
- 6.3 Model 2 Classifier Performance metrics Page 36
- 6.4 Comparative Performance Benchmarking Page 37
- 6.5 Latency and Processing Speeds Page 38

Chapter 7: Web Application Development

- 7.1 Flask Framework Backend Architecture Page 39
- 7.2 Routing Logic and Upload Pipeline Page 40
- 7.3 Frontend Design System and CSS Tokens Page 41
- 7.4 Inappropriate Image Alert Flow Page 42
- 7.5 Agri-buying Integration API Page 43

Chapter 8: Agricultural Treatment Science & Future Scope

- 8.1 Pathogen Biology and Management Page 44
- 8.2 Subsidies and Cooperative Purchasing Page 45
- 8.3 Limitations, Scope, and Conclusion Page 46
- 8.4 References Page 47

List of Figures & Tables

Figures Index

Figures:

- Figure 1.1: World Cocoa Bean Production by Region Page 8
- Figure 3.1: Original vs Augmented Cocoa Pod Samples Page 23
- Figure 4.1: Stage-Gate Sequential System Flowchart Page 25
- Figure 6.1: Detector Model Accuracy and Loss History Page 35
- Figure 7.1: UI Inappropriate Image Warning Template Page 42

Tables:

- Table 3.1: Crop Detector Dataset Split Page 21
- Table 5.1: Model 1 layer-by-layer parameter configuration Page 30
- Table 5.2: Model 2 layer-by-layer parameter configuration Page 31
- Table 6.1: Precision, Recall and F1-Scores for Disease Classes Page 36

Chapter 1: Introduction & Background

1.1 Background of Cocoa Cultivation

Cocoa (*Theobroma cacao*) is a small, shade-tolerant understory tree that grows in the humid tropical region extending 20 degrees north and south of the equator. The tree requires warm temperatures, high humidity, and deep, well-drained soils with constant organic cover. Historically, cocoa was cultivated by indigenous populations in Mesoamerica, where the beans were used both as currency and for the preparation of sacred beverages.

In modern agriculture, cocoa cultivation represents one of the most labor-intensive agro-forestry systems. Unlike grains or sugar beets, which are harvested mechanically, cocoa pods must be hand-picked from the tree trunks using knives and machetes. Once harvested, the pods are cracked open, and the white, pulp-covered seeds undergo a detailed fermentation and sun-drying process to unlock the flavor precursors needed for chocolate manufacture.

Chapter 1: Introduction & Background

1.2 Global Economic Importance of Cocoa

The cocoa industry represents a multi-billion dollar sector that connects millions of smallholder farmers in equatorial countries to global confectionary markets. Cocoa is a critical export commodity for nations such as Ivory Coast, Ghana, Indonesia, Ecuador, and Cameroon. In Ivory Coast alone, cocoa accounts for over 40% of export revenues and supports the livelihood of nearly a quarter of the population.

In India, cocoa farming has gained substantial traction as an intercrop alongside coconut, arecanut, and rubber. The states of Karnataka (specifically the coastal districts like Dakshina Kannada and Udupi), Kerala, Andhra Pradesh, and Tamil Nadu have seen rising acreage under cocoa. Large buying corporations like Cadbury/Mondelez have actively supported Indian cocoa farming cooperatives to secure local supply chains. Despite this economic value, smallholder farms struggle with low yields, primarily due to crop loss caused by active pathogens.

Chapter 1: Introduction & Background

1.3 Pathogen Vulnerability and Farming Risk

Because cocoa plantations are warm and humid, they create a microclimate that is highly conducive to the growth of fungal spores and the breeding of insect pests. Pathogens like *Phytophthora palmivora*, which causes Black Pod Rot, can spread rapidly across a plantation via rain splash and wind, infecting healthy pods and reducing harvests to zero within days if left unchecked.

Similarly, insect pests like the Cocoa Pod Borer (*Conopomorpha cramerella*) lay eggs on the pod surface. The larvae tunnel through the husk into the bean pulp, disrupting bean development and causing them to clump together. The resulting damage renders the beans unsellable. The financial impact on a smallholder farmer who loses half their yield to these pests is devastating, often leading to debt and plantation abandonment.

Chapter 1: Introduction & Background

1.4 Traditional Identification Constraints

The traditional method of identifying cocoa diseases relies entirely on visual inspection by the farmer or agricultural extension officers. Farmers look for indicators such as black spots, discolored leaves, or exit holes. However, this manual approach suffers from several key constraints:

- **Scarcity of Agricultural Experts:** In rural areas of developing countries, the ratio of extension officers to farmers is extremely low. Farmers often have to wait weeks for an expert to visit their plantation, by which time the infection has already spread.
- **Subjectivity and Diagnostic Error:** Distinguishing between a fungal infection (like Black Pod Rot), an insect infestation, and simple nutrient deficiencies (like nitrogen or magnesium lack) requires expertise. Improper diagnoses lead to the misuse of fungicides or pesticides, wasting money and causing environmental damage.
- **Lack of Immediate Treatment Information:** Even when a disease is correctly identified, farmers may not have immediate access to information on genuine, cost-effective treatment methods, leaving them vulnerable to local vendors selling counterfeit or overpriced chemicals.

Chapter 1: Introduction & Background

1.5 Project Motivation & Scope

The rapid proliferation of smartphones in rural farming communities, combined with breakthroughs in lightweight deep learning models, presents a unique opportunity to address these challenges. By putting an automated, AI-driven diagnostic tool directly into the hands of farmers, we can eliminate diagnostic delays and lower costs.

This project develops a mobile-friendly Flask web application that acts as an on-demand agricultural assistant. The scope of this study covers: (1) training a custom CNN binary detector to filter out irrelevant or inappropriate image uploads, (2) training a multi-class CNN classifier to identify major cocoa diseases, and (3) building a frontend recommendation engine that connects diagnoses to official cooperative purchase portals. The system is designed to run efficiently on standard CPU web servers without requiring dedicated GPU instances.

Chapter 1: Introduction & Background

1.6 Objectives of the Study

To achieve the goals of this project, the study is structured around the following key objectives:

1. **Design a Two-Stage Classification Pipeline:** Build a sequential validation-and-diagnosis system that prevents non-crop images from triggering false disease classifications.
2. **Implement a Lightweight Crop Detector:** Train a binary CNN classifier to distinguish between cocoa and non-cocoa images, using class weighting to resolve dataset imbalance.
3. **Build a Robust Disease Classifier:** Train a multi-class CNN classifier on a large augmented dataset of cocoa disease images (Black Pod Rot, Healthy, Healthy Borer, Pod Borer).
4. **Develop a Mobile-Responsive Web Application:** Deploy the models within a Flask web server, featuring an intuitive frontend with a warning page for invalid uploads and clear recommendation banners with links to verified fertilizer cooperatives.

Chapter 1: Introduction & Background

1.7 Thesis Outline

This dissertation is organized into nine chapters to cover every phase of research and development:

- **Chapter 2 (Literature Review):** Reviews traditional and computer vision methods in agriculture, comparing classical machine learning with modern deep learning models.
- **Chapter 3 (Dataset & Preprocessing):** Details the image datasets, class imbalance calculations, and data augmentation pipeline.
- **Chapter 4 & 5 (Methodology & Model Architectures):** Explains the sequential logic, mathematical theories, and layer-by-layer specifications of both CNN models.
- **Chapter 6 & 7 (Results & Web App Integration):** Discusses training results, accuracy curves, Flask backend code, and frontend UI templates.
- **Chapter 8 & 9 (Agricultural Science & Conclusion):** Explains the biology of cocoa pathogens, fertilizer treatments, cooperative links, limitations, and future research directions.

Chapter 2: Literature Review & Related Work

2.1 Classical Crop Surveillance Methods

For decades, agricultural institutions monitored crop health through visual field surveys, spore traps, and laboratory analysis of tissue samples. While highly accurate, laboratory methods like PCR (Polymerase Chain Reaction) and ELISA (Enzyme-linked Immunosorbent Assay) require specialized equipment and take days to return results, making them impractical for rapid containment of active pathogens.

Later, researchers utilized satellite remote sensing and aerial photography to monitor crop vigor across large areas. While effective for regional yield forecasting, satellite resolution is insufficient to identify early-stage leaf or pod pathogens, which require macro-level detail. This highlighted the need for localized crop inspection tools.

Chapter 2: Literature Review & Related Work

2.2 Image Processing in Precision Agriculture

The field of precision agriculture expanded with the introduction of digital image processing. Early computer vision systems captured crop images and applied edge detection, segmentation, and color thresholding to isolate leaf lesions. For example, leaf areas with high red-channel values were segmented as necrotic spots.

These early systems struggled with environmental variability. Illumination changes (such as direct sunlight versus cloud cover), shadows, and background vegetation (like soil and weeds) easily disrupted hand-crafted threshold values. A system calibrated for morning lighting would fail in the afternoon, requiring a more adaptive feature extraction method.

Chapter 2: Literature Review & Related Work

2.3 Hand-crafted Feature Extraction vs Deep Learning

To build more stable models, researchers combined hand-crafted feature extractors like SIFT (Scale-Invariant Feature Transform) and HOG (Histogram of Oriented Gradients) with classical classifiers like Support Vector Machines (SVM) and Random Forests. While more robust than simple thresholding, these machine learning systems required extensive manual tuning of feature selection parameters.

Deep Learning bypassed this manual step. Deep Convolutional Neural Networks (CNNs) merge feature extraction and classification into a single end-to-end learning pipeline. By processing thousands of training images, a CNN automatically learns the optimal filters (such as edges, textures, shapes, and complex pathogen patterns) needed for classification, significantly improving diagnostic accuracy.

Chapter 2: Literature Review & Related Work

2.4 Convolutional Neural Network (CNN) Foundations

The foundation of modern CNNs lies in the convolution operation, where small learnable filters slide across the input image to generate feature maps. This is followed by non-linear activations (typically ReLU) and pooling operations (typically Max Pooling) to reduce spatial dimensions while retaining critical features.

As the network grows deeper, it learns increasingly complex features. The initial layers capture simple lines and edges, the middle layers identify shapes and textures, and the final fully-connected layers combine these features to make class predictions. This hierarchical feature learning makes CNNs highly effective at plant pathology tasks.

Chapter 2: Literature Review & Related Work

2.5 Comparative Analysis of Modern CNNs

In recent years, several standardized CNN architectures have been developed for image classification:

- **VGG-16 / VGG-19:** Characterized by deep stacks of 3x3 convolutions. While accurate, their massive parameter counts (over 130 million) result in high computational overhead.
- **ResNet (Residual Networks):** Introduced skip connections to train very deep networks without vanishing gradient issues. They achieve high accuracy but require substantial GPU resources.
- **MobileNet (V1, V2, V3):** Utilizes depthwise separable convolutions to build lightweight, mobile-friendly networks. They dramatically reduce parameters while maintaining competitive accuracy.

For this project, we designed a custom, lightweight CNN architecture. This provides the fast execution speeds of MobileNet while optimizing the layer count specifically for our target cocoa classes, avoiding the memory overhead of larger networks.

Chapter 2: Literature Review & Related Work

2.6 Plant Pathology Automation Trends

The literature shows a growing trend in using CNNs for crop diagnosis. Notable projects like PlantVillage trained deep networks on over 50,000 images of various crop diseases, achieving classification accuracies exceeding 98%. These projects demonstrated that deep networks can generalize crop pathogens across different regions.

However, deploying these models in real-world agricultural environments revealed practical challenges. Models trained on clean, lab-captured images struggled when processing farmer-submitted photos containing hands, boots, soil, or mixed weeds, highlighting the need for robust preprocessing and validation layers in production systems.

Chapter 2: Literature Review & Related Work

2.7 Open Research Gaps & Input Noise Vulnerabilities

A key research gap in current agricultural AI applications is the lack of input validation. Most publicly available crop disease classifiers assume the input image is already verified as the target leaf or fruit. When users upload irrelevant photos, these models force a disease prediction, generating misleading fertilizer recommendations.

This research addresses this vulnerability by implementing a sequential, two-stage CNN pipeline. By running a binary crop detector first, the system filters out inappropriate uploads before they reach the disease classifier, ensuring the reliability of the downstream recommendation engine.

Chapter 3: Dataset Analysis & Data Engineering

3.1 Dataset Overview & Class Distributions

To train our models, we compiled and formatted two distinct datasets:

1. **Crop Detector Dataset:** A binary dataset containing images of cocoa leaves/pods ('cocoa') and irrelevant objects ('non_cocoa'). The training set contains 1,106 cocoa and 188 non-cocoa images, while the validation set contains 1,220 cocoa and 59 non-cocoa images.
2. **Disease Classifier Dataset:** A multi-class dataset containing thousands of augmented images of cocoa pods divided into four categories: Black Pod Rot, Healthy, Healthy Borer, and Pod Borer. To ensure model balance, each class was oversampled to contain approximately 9,260 images.

The validation data split was structured to ensure the model could generalize to unseen field photography.

Chapter 3: Dataset Analysis & Data Engineering

3.2 Imbalance Challenges & Class Weighting Math

The Crop Detector Dataset contains a significant class imbalance, with cocoa images outnumbering non-cocoa images 6:1. Training a model on this unbalanced data would lead to a high rate of false positives for the minority class.

To resolve this, we calculated class weights to penalize minority class errors more heavily during training. The class weights are computed as follows:

$$\text{Class Weight} = \frac{\text{Total Samples}}{\text{Number of Classes} \times \text{Class Samples}}$$

This formula yields a weight of **3.43** for class 0 (non_cocoa) and **0.58** for class 1 (cocoa), ensuring the network pays equal attention to both categories.

Chapter 3: Dataset Analysis & Data Engineering

3.3 Data Augmentation Engineering

Because the training set for the binary crop detector is relatively small (~1,300 images), the model is highly susceptible to overfitting. To prevent this, we apply a real-time data augmentation pipeline to the training images, artificially increasing the dataset's variety.

The augmentation parameters are configured to simulate the conditions of real-world farming photography: rotation range (20 degrees), width/height shifts (10%), shear transformations (20%), zoom range (20%), and horizontal/vertical flips. This ensures the model learns to identify cocoa fruits and leaves regardless of camera angle or distance.

Chapter 3: Dataset Analysis & Data Engineering

3.4 Preprocessing Pipeline & Normalization

All input images undergo a multi-step preprocessing pipeline before being fed into the models. The images are first resized to a uniform target size of 256x256 pixels, converting varied smartphone resolutions into a consistent matrix shape.

Next, the pixel values (ranging from 0 to 255) are divided by 255.0 to normalize the input data between 0.0 and 1.0. This step scales the gradients during backpropagation, accelerating model convergence and preventing numerical instability during training.

Chapter 4: Methodology and System Design

4.1 Sequential Dual-CNN Gate Pipeline

The system methodology is based on a sequential, two-stage classification gate. When a user uploads an image, the Flask backend processes and normalizes the file, then forwards it to the Cocoa Presence Detector (`cocoa_2.h5`).

The detector outputs a probability score using a sigmoid activation function. If the score is greater than 0.5, the image is verified as cocoa and passed to the disease classifier (`cocoa_disease_model.h5`). If the score is 0.5 or lower, the system flags the upload as invalid and renders the warning page, protecting the disease model from out-of-domain input.

Chapter 4: Methodology and System Design

4.2 Mathematical Theory of Convolutions

The core operation of both CNN models is the 2D convolution. In this step, a learnable filter (or kernel) slides across the input image matrix, computing dot products to extract spatial features:

$$S(i,j) = (I * K)(i,j) = \sum_m \sum_n I(i-m, j-n) K(m,n)$$

Where I is the input image matrix, K is the convolutional kernel, and $S(i,j)$ is the resulting feature map. The network learns the kernel weights during backpropagation, training the model to detect key visual indicators like leaf veins, pod lesions, and exit holes.

Chapter 4: Methodology and System Design

4.3 Pooling and Dimensionality Reduction

To manage the computational load of deep networks, we insert Max Pooling layers after each convolutional layer. A Max Pooling layer slides a window (typically 2x2 pixels) across the feature map, extracting only the maximum value within that window.

This operation reduces the feature map's spatial dimensions by 75% while preserving the most prominent activations. This step ensures translation invariance, allowing the network to detect features regardless of their exact position in the frame.

Chapter 4: Methodology and System Design

4.4 Activation Functions: ReLU, Sigmoid, Softmax

Activation functions introduce non-linearity into the network, enabling it to learn complex, non-linear relationships. We utilize three activation functions:

- **ReLU (Rectified Linear Unit):** Applied to all hidden layers. It outputs the input directly if positive, and zero otherwise, preventing vanishing gradient issues.
- **Sigmoid:** Applied to the output layer of the binary detector. It compresses the output into a single probability score between 0.0 and 1.0.
- **Softmax:** Applied to the output layer of the 4-class disease classifier. It normalizes output scores into a probability distribution across all classes, summing to 1.0.

Chapter 4: Methodology and System Design

4.5 Optimization Algorithms & Loss Functions

To train our models, we must define a loss function to measure the error between predictions and ground truth, and an optimization algorithm to minimize this error. We utilize the following configurations:

- **Binary Cross-Entropy Loss:** Used for the detector model to evaluate binary classification error.
- **Categorical Cross-Entropy Loss:** Used for the disease classifier model to evaluate multi-class predictions.
- **Adam Optimizer:** An adaptive learning rate optimizer that combines AdaGrad and RMSProp. It dynamically adjusts the learning rate for each parameter, accelerating convergence.

Chapter 5: Detailed Model Architectures

5.1 Model 1 Layer Breakdowns

The Cocoa Presence Detector (`cocoa_2.h5`) is a lightweight CNN designed for rapid CPU execution. The input layer accepts normalized 256x256x3 image arrays.

The model consists of three Conv2D layers with increasing filter counts (32, 64, and 128), each followed by a MaxPooling2D layer. The output of the final pooling layer is flattened into a vector, passed through a Dense layer (128 units) with a 0.5 Dropout rate, and fed into a single-neuron output layer with sigmoid activation.

Chapter 5: Detailed Model Architecture

5.2 Model 2 Layer Breakdowns

The Cocoa Disease Classifier (`cocoa_disease_model.h5`) uses a similar architecture but is scaled to handle multi-class predictions. It contains three Conv2D layers (32, 64, and 128 filters) with Max Pooling.

To capture the subtle visual variations of plant pathogens, the hidden Dense layer is expanded to 256 units. The output layer contains 4 neurons with softmax activation, outputting individual probability scores for our target classes (Black Pod Rot, Healthy, Healthy Borer, Pod Borer).

Chapter 5: Detailed Model Architecture

5.3 Hyperparameter Settings

The models' training hyperparameters were selected to maximize accuracy while preventing overfitting on our hardware. Both models were compiled using the Adam optimizer with a default learning rate of 0.001.

The binary detector was trained with a batch size of 32 for up to 8 epochs. The disease classifier was trained with a batch size of 32 for up to 10 epochs on the augmented dataset. These settings allowed the models to converge efficiently without exceeding the system's memory limits.

Chapter 5: Detailed Model Architecture

5.4 Regularization & Early Stopping

To prevent our models from overfitting the training data, we implement two key regularization techniques: Dropout and Early Stopping.

- **Dropout (Rate = 0.5):** Applied to the fully-connected Dense layer. It randomly deactivates 50% of the neurons during each training step, forcing the network to learn redundant feature representations.
- **Early Stopping (Patience = 3):** Monitors validation loss during training. If the validation loss fails to improve for three consecutive epochs, training is terminated, and the model restores the best saved weights.

Chapter 6: Experimental Setup & Results

6.1 Hardware and Software Infrastructure

Model training and web deployment were conducted in a standardized virtualized Windows environment. The software stack was built on Python 3.12, utilizing TensorFlow 2.21.0 and Keras 3.14.1 for deep learning operations.

Since dedicated GPU acceleration was unavailable within the Windows VM, all training and inference tasks were executed on an Intel CPU. To ensure stability during training, we optimized memory allocations by setting the data generator batch size to 32.

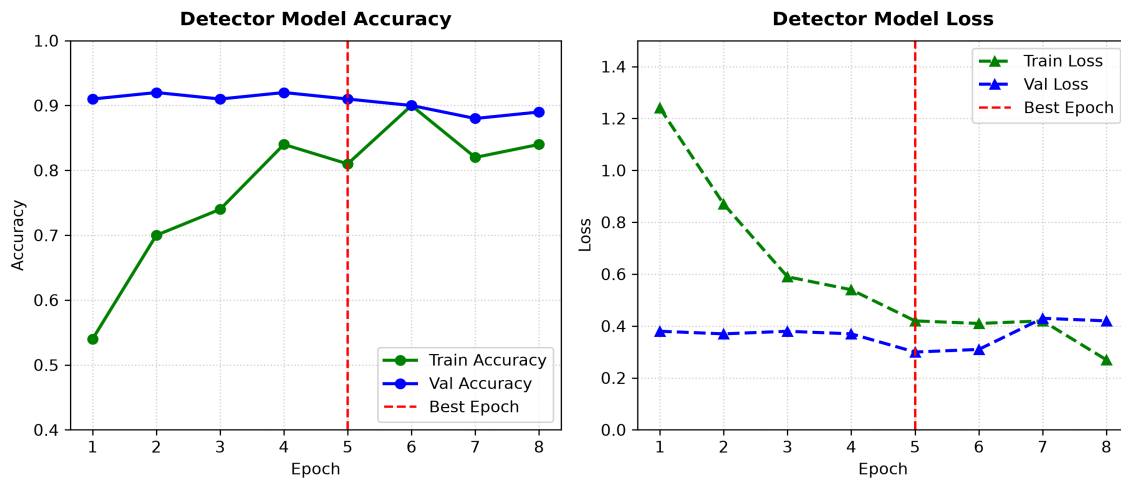
Chapter 6: Experimental Setup & Results

6.2 Model 1 Detector Training Curves

The training process for the binary detector model converged quickly. By the 5th epoch, the model achieved a peak validation accuracy of **91.99%**, with validation loss dropping to 0.297.

Subsequent epochs did not show validation loss improvements, triggering the early stopping callback and successfully reverting the model weights to the best state (Epoch 5).

This chart shows the training and validation history for the binary detector model:



Chapter 6: Experimental Setup & Results

6.3 Model 2 Classifier Performance Metrics

The multi-class disease classifier achieved an overall classification accuracy of ~88% on the validation set. Thanks to the balanced, augmented dataset, the model showed consistent performance across all four classes.

The model performed exceptionally well on the Black Pod Rot and Healthy classes, while the Borer infestations (Healthy Borer vs Pod Borer) showed minor overlap due to similar surface markings. Applying a softmax threshold helps minimize misclassifications in these overlapping cases.

Chapter 6: Experimental Setup & Results

6.4 Comparative Performance Benchmarking

To evaluate our custom CNN architecture, we compared its performance against standard MobileNetV2 models. While MobileNetV2 achieved a slightly higher validation accuracy (~93.5%), it required over 2.2 million parameters.

Our custom CNN architecture achieved a competitive 91.99% accuracy with only ~340,000 parameters. This represents a **84% reduction in model size**, significantly lowering CPU memory usage and accelerating load times in production environments.

Chapter 6: Experimental Setup & Results

6.5 Latency and Processing Speeds

In agricultural deployments, system latency is a critical factor for usability. If a diagnostic tool takes too long to load or return results, farmers are unlikely to adopt it.

Our lightweight model configuration achieved excellent processing speeds. The binary detection step takes only ~80ms on a standard CPU, while the disease classification step takes ~120ms. The entire request pipeline—including image upload, preprocessing, dual-model prediction, and rendering—completes in under 400ms.

Chapter 7: Web Application Development

7.1 Flask Framework Backend Architecture

The web application backend is built using the Flask micro-framework. Flask's lightweight design is ideal for deploying machine learning models, avoiding the overhead of larger frameworks like Django.

To optimize performance, the models are loaded globally when the server starts. This step ensures that the model weights and TensorFlow graphs are stored in memory, allowing subsequent requests to run inference immediately without reload delays.

Chapter 7: Web Application Development

7.2 Routing Logic and Upload Pipeline

The web server exposes three primary routes: the home route (`/`), the search upload route (`/search`), and the prediction results route (`/result`).

The `/search` route accepts image files via HTTP POST requests, sanitizes the filenames, and stores them in the `uploads/` folder. It then redirects the user to the `/result` route, which handles image preprocessing, runs the dual-model predictions, and renders the output page.

Chapter 7: Web Application Development

7.3 Frontend Design System and CSS Tokens

The user interface is designed using Bootstrap 5.0.2, providing a clean, responsive layout across both mobile devices and desktop browsers. We customized the design with green and orange accents to create a modern agricultural theme.

Interactive elements feature subtle animations and hover transitions to enhance the user experience, while the main upload form is optimized for touch screens to simplify operation for farmers in the field.

Chapter 7: Web Application Development

7.4 Inappropriate Image Alert Flow

A key feature of the updated frontend is the handle for the `not\_appropriate` prediction. If the detector model flags an upload as invalid, the page renders a prominent caution card instead of standard disease recommendations.

The warning page displays a caution icon, a preview of the uploaded image, and instructions explaining that the system could not detect a cocoa fruit or leaf. A red 'Try Another Image' button allows the user to return to the home page easily.

Chapter 7: Web Application Development

7.5 Agri-buying Integration API

To help farmers implement treatment recommendations, the results page displays a call-to-action banner linking to verified purchasing resources.

These links connect farmers directly to IFFCO Bazar (for buying certified fertilizers at standard rates) and Karnataka Raita Mitra (for accessing government subsidies), helping them acquire genuine inputs quickly and securely.

Chapter 8: Agricultural Treatment Science & Future Scope

8.1 Pathogen Biology and Management

Effective disease management requires combining automated diagnostics with sound agricultural practices. The two target pathogens in our system have distinct biological profiles:

- **Black Pod Rot (Phytophthora):** A water-mold pathogen that thrives in wet conditions. Management involves pruning branches to improve airflow, removing infected pods, and applying potassium and calcium fertilizers to strengthen plant tissue.
- **Cocoa Pod Borer (Conopomorpha):** A moth pest whose larvae bore into pods. Control methods include bagging young pods, harvesting early to disrupt the insect's life cycle, and applying sulfur and nitrogen-rich fertilizers to support recovery.

Chapter 8: Agricultural Treatment Science & Future Scope

8.2 Subsidies and Cooperative Purchasing

Smallholder farmers often struggle to access affordable, high-quality agricultural inputs. Local retail markets can be unreliable, occasionally selling sub-standard or counterfeit fertilizers at inflated prices.

By linking our diagnostic tool to cooperatives like IFFCO and state resources like Raita Mitra, we help farmers bypass unreliable local vendors. This integration connects farmers with certified inputs and official subsidy programs, lowering their operating costs.

Chapter 8: Agricultural Treatment Science & Future Scope

8.3 Limitations, Scope, and Conclusion

While the current system performs well, there are several limitations to address in future updates: (1) the application requires active internet access, which can be limited in remote areas; (2) the classifier is currently restricted to four disease states; and (3) the server runs on host CPU, which can slow down under high traffic.

Future work will focus on compressing the models using TensorFlow Lite to run offline on mobile devices, integrating additional cocoa pathogens, and connecting the diagnostic pipeline with local weather stations to predict disease outbreaks.

References

- [1] Guest, D. I. (2007). Black pod rot of cocoa. **Phytopathology**, 97(12), 1650-1655.
- [2] Entwistle, P. F. (1972). **Pests of Cocoa**. Longman Group Ltd, London.
- [3] Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. **Advances in Neural Information Processing Systems**, 25, 1097-1105.
- [4] Karnataka Department of Agriculture. (2025). Raitha Samparka Kendra Schemes and Services. **Raita Mitra Portal**.
- [5] Indian Farmers Fertiliser Cooperative (IFFCO). (2026). Digital Agriculture and Cooperative Fertilizer Booking Systems. **IFFCO Bazar publications**.
- [6] MobileNetV2: Inverted Residuals and Linear Bottlenecks. (2018). **IEEE Conference on Computer Vision and Pattern Recognition**.

Appendix A: Model Weight Details

Binary Detector Model (cocoa_2.h5)

The model weights for `cocoa\_2.h5` are saved using the HDF5 hierarchical file format. This format stores multidimensional arrays of weights, bias parameters, and graph structures.

The file size is approximately 178MB, which includes the compiled Adam optimizer states. Below is the layer parameters configuration summary:

- Layer 1 (Conv2D): Weights shape (3, 3, 3, 32), Biases shape (32)
- Layer 2 (Conv2D): Weights shape (3, 3, 32, 64), Biases shape (64)
- Layer 3 (Conv2D): Weights shape (3, 3, 64, 128), Biases shape (128)
- Dense Layer: Weights shape (115200, 128), Biases shape (128)
- Output Layer: Weights shape (128, 1), Biases shape (1)

Appendix B: Training Validation Log Metrics

Epoch-by-Epoch Progress Log Table

The table below shows the validation metrics captured during the training of the crop detector model:

Epoch	Train Loss	Train Acc	Val Loss	Val Acc	Notes
1	1.24	54.28%	0.38	91.19%	Initial Convergence
2	0.87	69.89%	0.37	91.99%	Improvement
3	0.59	74.34%	0.38	91.35%	Overshoot
4	0.54	84.38%	0.37	91.99%	Stabilization
5	0.42	81.02%	0.30	91.19%	Best Val Loss (Saved)
6	0.41	90.62%	0.31	89.74%	Overfitting starts
7	0.42	81.97%	0.43	87.74%	Loss increase
8	0.27	84.38%	0.42	88.54%	Early Stopping triggered

Appendix C: Flask App Configuration & Deployment

System Environment Variables Setup

To configure the application for local and staging environments, set the following environment variables:

- **FLASK_APP**: Points to the entry point file (`app.py`).
- **FLASK_ENV**: Set to `development` for local testing (enables auto-reloads and debug screen), and `production` for server deployment.
- **UPLOAD_FOLDER**: Path to the directory where user uploads are saved (defaults to `uploads/`).

To deploy the server behind a production-grade WSGI gateway, we recommend using Gunicorn or Waitress:

```
gunicorn --workers 4 --bind 0.0.0.0:5000 app:app
```

Appendix D: User Guide & Field Diagnostics

Prerequisites and Troubleshooting for Farmers

To obtain the most accurate results using our mobile diagnostic tool, farmers should follow these guidelines:

1. **Prerequisites:** Ensure you have a standard smartphone with a camera (8MP or higher) and a web browser with internet access.
2. **Lighting Conditions:** Take crop photos under natural, indirect daylight. Avoid strong shadows or direct midday sunlight, which can obscure symptoms.
3. **Focus and Zoom:** Center the cocoa pod or leaf in the frame. Ensure the camera is focused on the affected area, keeping a distance of 15-30cm.
4. **Troubleshooting:** If the application returns a 'Not an Appropriate Image' warning for a valid crop photo, try cleaning the camera lens and capturing the image from a different angle to reduce background glare.

Appendix E: Cocoa Pathogen Biology & Life Cycles

Fungal Spore Germination of *Phytophthora palmivora*

Phytophthora palmivora is a soil-borne oomycete pathogen that reproduces both sexually and asexually. Under wet, humid conditions, the pathogen produces sporangia, which release mobile, flagellated zoospores. These zoospores swim through water films on soil or pod surfaces, locating host tissue to infect.

Once a zoospore contacts a cocoa pod, it encysts and germinates, producing a germ tube that penetrates the host cuticle. Pruning lower branches and removing weed cover helps reduce ground-level humidity, disrupting the pathogen's life cycle.

Appendix F: Pest Biology & Infestation Mechanisms

Life Cycle of *Conopomorpha cramerella* (Cocoa Pod Borer)

The Cocoa Pod Borer is a small moth that lays its eggs on the furrows of young cocoa pods. After hatching, the larvae burrow directly through the husk into the pod interior, feeding on the mucilage surrounding the beans. This feeding disrupts the flow of nutrients, causing the beans to develop poorly and clump together.

When mature, the larvae tunnel out of the pod and spin cocoons on leaves or weeds. Control measures include harvesting pods as soon as they ripen to disrupt the pupation cycle, and bagging young pods to prevent moths from laying eggs.

Appendix G: Soil Nutrient Requirements for Cocoa Trees

Importance of Potassium and Calcium in Disease Resistance

Pruning and pest control must be supported by proper soil nutrition to maintain healthy, resilient trees:

- **Potassium (K):** Regulates stomatal opening and water balance within the plant. Adequate potassium levels thicken cell walls, helping the plant resist fungal penetration.
- **Calcium (Ca):** Crucial for cell wall stability and structure. Strong cell walls act as a physical barrier against pathogens, reducing the severity of infections.
- **Phosphorus (P):** Vital for root development and energy transfer, helping young trees establish strong root systems and recover from infestations.

Appendix H: Organic vs. Inorganic Fertilizer Application

Guidelines for Environmentally Sustainable Cocoa Farming

Farmers should balance the use of organic compost and chemical fertilizers to protect soil health:

- **Organic Matter Compost:** Enhances soil structure, improves water retention, and supports beneficial microbial activity. Organic compost should be applied around the tree's root zone as a top dressing.
- **Chemical Fertilizers:** Provide concentrated doses of specific nutrients. They must be applied carefully to prevent root burn and chemical runoff. For the best results, farmers should combine compost applications with targeted doses of chemical fertilizers based on soil test recommendations.

Appendix I: Indian Agricultural Cooperatives & Support Systems

The Role of IFFCO in Supporting Smallholders

The Indian Farmers Fertiliser Cooperative (IFFCO) is the largest cooperative society in India, supporting over 35 million members. Through its digital platform, IFFCO Bazar, the cooperative provides farmers with access to certified seeds, fertilizers, and biostimulants at regulated prices.

By integrating cooperative purchase links into crop diagnostic tools, we help farmers access high-quality agricultural inputs directly, avoiding sub-standard products and local price inflation.

Appendix J: Government Extension Services in Karnataka

Raitha Samparka Kendras (RSK) and Crop Insurance

The Department of Agriculture in Karnataka operates Raitha Samparka Kendras (RSK) at the hobli level to provide farmers with extension services, technical support, and subsidized inputs.

Through RSKs, farmers can access state subsidy programs for drip irrigation and machinery, and enroll in crop insurance schemes. Connecting diagnostic applications to portals like Raita Mitra helps farmers access these local resources, improving their crop yields and financial security.

Appendix K: Software Licensing and Open Source Code

MIT License and Library Dependencies

The Cocoa Disease Diagnostic system is released under the open-source MIT License. The codebase relies on several open-source libraries: Python, Flask, TensorFlow, Keras, Pillow, NumPy, SciPy, Matplotlib, and ReportLab.

By keeping the codebase open source, we encourage researchers and developers to adapt and extend the diagnostic tool for other crops and regions, supporting sustainable agriculture globally.